UNIVERSIDAD DEL VALLE DE GUATEMALA

Deep Learning

Ing. Kevin Recinos



Proyecto 3

Text-to-SQL: fine-tuning de un LLM con LoRA

Cindy Mishelle Gualim Perez

Gadiel Ocaña

GUATEMALA, 25 de octubre de 2026

1. Objetivo y alcance

El objetivo es ajustar con LoRA un modelo pequeño para que, dado el esquema de una base de datos en el formato que devuelve `SHOW CREATE TABLE` y una pregunta en español, genere una consulta MySQL correcta. El criterio de acierto es la exactitud por ejecución: se comparan los resultados de ejecutar la consulta generada y la de referencia, nunca su texto.

Este informe documenta el dataset, la validación propia, el baseline medido, las dos configuraciones de LoRA, el experimento de inferencia, el análisis de errores y la predicción razonada del desempeño en la evaluación oculta. Los números que aparecen fueron medidos ejecutando el código del repositorio, salvo donde se indica explícitamente que un resultado está pendiente.

2. Construcción del dataset

2.1 Esquemas propios

Se escribieron 12 esquemas de 6 a 7 tablas, con `COMMENT` en cada tabla y cada columna, porque es exactamente lo que el evaluador entrega al modelo. La mezcla es deliberada: cinco en español, tres en inglés, dos con nombres abreviados (`pac`, `med`, `mbr`, `cls`) y dos mezclados, uno con clave primaria no numérica y otro con tabla puente. El enunciado anuncia esas mismas condiciones para la evaluación oculta.

Los datos no se escribieron a mano: un poblador genérico lee `information_schema`, ordena las tablas topológicamente por sus claves foráneas y genera valores con semilla fija según el tipo de cada columna y pistas de su nombre. Mantiene coherencia cronológica, respeta restricciones `UNIQUE` y acota cada valor al rango del tipo declarado. El resultado son 78 tablas y 9,525 filas reproducibles.

2.2 Generación y validación de los pares

Se escribieron 32 plantillas paramétricas repartidas en los tres niveles de la evaluación: 11 fáciles de una sola tabla, 10 medias con `JOIN`, `GROUP BY` y `HAVING`, y 11 difíciles con subconsultas correlacionadas, `WITH`, funciones de ventana y fechas. Los umbrales de los filtros no son constantes inventadas: se muestrean 80 filas de cada tabla y se usan los cuantiles p25, p50 y p75, de modo que las consultas devuelvan resultados no vacíos.

Los 2,464 pares generados se ejecutaron en MySQL 8.4.10 con un usuario de solo lectura, replicando la condición del evaluador.

Resultado	Pares	Porcentaje
Generados	2,464	100%
Válidos	2,391	97.0%
Descartados por resultado vacío	73	3.0%
Descartados por error de sintaxis	0	0%
Descartados por ambigüedad de la pregunta	0	0%

Tabla 1. Validación por ejecución de los pares generados. El detalle por par está en datos/descartes.csv.

Se descartan los pares cuyo resultado es vacío por una razón concreta: con cero filas, la comparación por ejecución no distingue una consulta correcta de una equivocada, porque cualquier error que también devuelva vacío cuenta como acierto. Quedan 2,320 pares de entrenamiento (894 fáciles, 862 medios, 564 difíciles) y 71 reservados para vigilar la pérdida durante el entrenamiento.

2.3 Ambigüedad detectada y corregida

La verificación de preguntas duplicadas no encontró repetición sino ambigüedad real. La tabla `vuelos` tiene dos claves foráneas a `aeropuertos` (`origen` y `destino`), y `prestamos` apunta a `clientes` y a `sucursales`, que además tienen ambas una columna `nombre`. El problema no era el par descartado sino el que se conservaba: entrenaba al modelo a adivinar. Se añadió desambiguación automática y se corrigieron cuatro redacciones que omitían el nombre de la tabla. Las preguntas ambiguas pasaron de 19 a 13 y finalmente a 0.

3. Validación propia

Se construyeron dos esquemas que no aparecen en el entrenamiento, `veterinaria` (español) y `hotel` (inglés), y se escribieron a mano 40 preguntas con su consulta de referencia: 13 fáciles, 13 medias y 14 difíciles, con peso total 81. Las preguntas están en español también sobre el esquema en inglés. Cinco exigen un orden de filas explícito; el resto se compara como conjunto.

La partición es por esquema, no por pregunta: la intersección entre las bases de entrenamiento y las de validación es vacía, y no comparten ni un solo nombre de tabla. Las 40 consultas de referencia se verificaron ejecutándolas; todas corren y devuelven filas.

Escribir las preguntas a mano reveló dos defectos del poblador que ninguna prueba automática habría encontrado: las mascotas recibían fechas de nacimiento de persona, y la calificación de las reseñas llegaba a 100 cuando su `COMMENT` indica un rango de 1 a 5.

4. Baseline: el modelo sin adapter

El baseline es el mismo `solucion.py` con `usar_adapter: false`, es decir, el modelo base con exactamente el mismo prompt. Se evaluó sobre las 40 preguntas de la validación propia.

Nivel	Preguntas acertadas	Peso acertado	EX del nivel
Fácil (peso 1)	12 / 13	12 / 13	0.92
Media (peso 2)	10 / 13	20 / 26	0.77
Difícil (peso 3)	5 / 14	15 / 42	0.36
Total	**27 / 40**	**47 / 81**	**0.580**

Tabla 2. EX ponderada del modelo base sobre la validación propia.

El resultado es más alto de lo que anticipábamos y tiene una consecuencia directa sobre la nota: el enunciado otorga 10 puntos si el adapter mejora la EX en al menos 0.15, y esa mejora se mide contra una base que ya está en 0.580. El margen por arriba es estrecho.

Ocho de las 40 consultas generadas ni siquiera ejecutan. El tiempo medido fue de 26.8 segundos por pregunta con un máximo de 76.1, pero esa medición se hizo en CPU; el contrato de 30

segundos debe verificarse en el entorno de evaluación, una GPU T4, donde se espera un orden de magnitud menos.

5. Configuraciones de LoRA

Se definieron dos configuraciones que difieren en un solo factor, el rank. Todo lo demás se mantiene fijo: `alpha=32`, `dropout=0.05`, `lr=2e-4`, dos épocas, lote efectivo 16, planificador coseno, semilla 2026 y los mismos siete módulos objetivo. Mantener `alpha` constante es intencional: escalarlo con el rank cambiaría a la vez la capacidad del adapter y la magnitud de su contribución, y la diferencia no se podría atribuir a ninguno de los dos.

	Configuración A	**Configuración B**
Rank	8	32
Parámetros entrenables	pendiente	pendiente
Tiempo de entrenamiento	pendiente	pendiente
Pérdida final (dev)	pendiente	pendiente
EX en validación propia	pendiente	pendiente

Tabla 3. Las dos configuraciones. El entrenamiento requiere GPU y se ejecuta con notebooks/02_entrenamiento.ipynb en Google Colab.

Hipótesis, escritas antes de entrenar. La configuración A, con unos 9 millones de parámetros entrenables, debería bastar para aprender el formato de salida y el anclaje al esquema, que es el error dominante del modelo base. La configuración B, con cuatro veces esa capacidad, solo debería ganar en el nivel difícil, y con mayor riesgo de sobreajuste a las 32 plantillas. La señal a vigilar es la pérdida de validación: si A y B coinciden, la capacidad extra no se está usando. Ante un empate se entrega A.

6. Experimento de inferencia

Se comparan tres variantes sobre el mismo adapter y el mismo criterio de EX, cambiando únicamente cómo se arma la inferencia: la directa, una con tres ejemplos resueltos en el prompt (few-shot) sobre un esquema inventado que no pertenece a ninguna partición, y una con autocorrección.

La autocorrección usa la función `ejecutar(sql)` que ofrece el evaluador. El ciclo ejecuta la consulta; si corre y devuelve filas, la entrega; si falla, vuelve a generar agregando al prompt la consulta anterior y el error del motor. Una consulta que se ejecuta pero devuelve cero filas se guarda como respaldo antes que entregar una rota. El contador de llamadas vive fuera del modelo, de modo que el tope de tres por pregunta se respeta siempre.

Hipótesis. La autocorrección debería producir la mayor ganancia, concentrada en los errores de columna y tabla inexistente, que son los únicos que el motor reporta de forma explícita y accionable. No esperamos ninguna mejora en los casos de resultado distinto: ahí la consulta corre sin error y el modelo no recibe señal alguna de que está equivocada. Ese es el límite duro del método. El few-shot debería aportar poco después del ajuste fino; si aportara mucho, sería señal de

que el adapter no aprendió el formato.

7. Análisis de errores

Los fallos se clasifican automáticamente en siete categorías. Las de sintaxis, tabla y columna inexistente se deducen del código de error de MySQL; las de join, agregación y filtro se deducen comparando la forma de la consulta generada con la de referencia, es decir, el conjunto de tablas, la presencia de agregados y las condiciones.

Categoría	Preguntas	Peso perdido	% del peso perdido
Columna inexistente	5	12	35%
Join	3	8	24%
Agregación	3	8	24%
Tabla inexistente	1	3	9%
Sintaxis	1	3	9%
Total	**13**	**34**	**100%**

Tabla 4. Errores del baseline sobre la validación propia. Se reporta el peso perdido porque es lo que mueve la métrica: fallar una pregunta difícil cuesta lo mismo que fallar tres fáciles.

El 44% del peso perdido corresponde a anclaje al esquema: columna y tabla inexistente suman 15 de los 34 puntos. Un ejemplo representativo es la pregunta 38, donde el modelo escribió `JOIN payments` sobre la base `hotel`, que no tiene ninguna tabla con ese nombre: la inventó a partir de lo que sabe de otros esquemas en lugar de leer el que tenía delante. No es un error de razonamiento sino de anclaje, y es precisamente lo que 2,320 ejemplos con el esquema en el prompt pueden corregir.

8. Predicción justificada de la EX oculta

Predicción: EX entre 0.45 y 0.65, con 0.55 como valor más probable. Mejora sobre el modelo sin adapter de +0.08 a +0.19.

El razonamiento parte del baseline medido, 0.580 sobre nuestra validación. A ese número le descontamos alrededor de 0.15 para la evaluación oculta, porque nuestras preguntas las escribimos nosotros sobre esquemas que diseñamos nosotros, y el docente no conoce nuestras convenciones de redacción.

El techo optimista del adapter se puede calcular: si corrigiera todo el anclaje al esquema, recuperaría los 15 puntos de peso de columna y tabla inexistente, lo que llevaría la EX de 0.580 a 0.765 y daría una mejora de +0.185. Es un techo, no una expectativa: lo realista es recuperar entre la mitad y dos tercios de esos puntos, más algo de los errores de agregación, donde el formato consistente del dataset debería ayudar con `ONLY_FULL_GROUP_BY`.

La limitación más seria que asumimos es que el dataset proviene de 32 plantillas y no de lenguaje natural libre. El modelo verá formas de preguntar que no aparecen en el entrenamiento. Lo mitigamos con dos paráfrasis por plantilla y alternando el nombre literal de la columna con la

etiqueta tomada de su `COMMENT`, pero no sustituye la variedad de preguntas escritas por personas distintas.

De la *diferencia* entre las dos versiones estamos más seguros que del valor absoluto, porque se mide sobre las mismas preguntas y absorbe el factor de redacción. Aun así, el umbral de +0.15 del enunciado queda dentro del rango pero no garantizado, y esa es la parte más expuesta de la entrega.

9. Reproducibilidad

Todo el pipeline es reproducible con semilla fija 2026: el poblador de datos, la generación de pares, la partición interna y la generación del modelo, que usa `do_sample=False` y un solo haz. Dos funciones críticas tienen pruebas unitarias, porque de ellas depende todo lo que se reporta: la limpieza de la salida del modelo (11 casos) y el criterio de exactitud por ejecución (20 casos).

Secuencia completa:

```
bash scripts/mysql_setup.sh docker
python scripts/cargar_esquemas.py --dir datos/esquemas --poblar
python scripts/generar_pares.py --limite 20
python scripts/validar_pares.py
python scripts/entrenar.py --etiqueta r8 --rank 8 --alpha 32
python scripts/evaluar.py --etiqueta baseline --sin-adapter
python scripts/analizar_errores.py
```

10. Licencias y uso de IA

El modelo base Qwen2.5-Coder-1.5B-Instruct se distribuye bajo licencia Apache 2.0. El código de este repositorio se publica bajo licencia MIT. El dataset es propio: los 12 esquemas, los datos y los 2,464 pares se generaron para este proyecto, por lo que no aplican las licencias de Spider ni de BIRD.

Se utilizó Claude (Anthropic) como asistente de programación en la redacción del DDL, la implementación de los scripts y la redacción de los documentos. Las decisiones de diseño documentadas en cada archivo fueron tomadas y revisadas por los integrantes. Las 40 preguntas de validación y sus consultas de referencia se escribieron a mano, no se generaron con un modelo, y cada una se verificó ejecutándola. Ninguna IA externa participa en la generación de la consulta que se evalúa: el único modelo que interviene en `solucion.py` es el Qwen permitido por el enunciado, con su adapter.

Anexo A. Repositorio

El código, el dataset, los notebooks y los resultados están en el repositorio privado compartido con el docente:

https://github.com/CindyGualim/deep-proyectoSQL

El historial completo, con autor y fecha de cada cambio, está en `historial_git.txt`, generado con `git log --all --date=iso --stat`. La bitácora fechada está en `BITACORA.md` y el reparto de tareas en `CONTRIBUCIONES.md`.

Anexo B. Entregables

Archivo	Contenido
solucion.py	Contrato de entrega: generar_sql(esquema, pregunta, ejecutar)
config.json	Modelo base, adapter, semilla
requirements.txt	Dependencias verificadas para Colab T4
adapter/	Adapter LoRA entrenado
datos/entrenamiento.jsonl	2,320 pares validados en MySQL 8.4.10
datos/descartes.csv	73 pares descartados con su motivo
datos/validacion/	2 esquemas y 40 preguntas propias
notebooks/	Datos, entrenamiento y evaluación
resultados/validacion_propia.csv	Una fila por pregunta y por corrida
resultados/analisis_errores.csv	Fallos clasificados por categoría
presentacion.pdf	12 diapositivas